

AOC2 Project 1 - Atomic instruction using in-memory processing and hazard management in a MIPS-based SoC

Javier Resano, José Luis Briz

Submission deadline: 31-03-2025 2025, 07:55 a.m. subject to confirmation via Moodle.

README

- **Save time by carefully reading this Guide.**
 - You will understand key issues still possibly unclear to you despite class theory and problems
 - It will help you organize and write your report.
 - **Do not change the name of the components and signals** used in the provided VHDL source code without checking it out with us.
 - All materials (source files, slides, problems, guides...) are under the Intellectual Property Rights of the University of Zaragoza. **The violation of these rights can entail legal consequences.**
 - **Language issues** - AOC2 has been declared an ELF (*English Language Friendly*) course. The main reference books and most in-house written materials, are delivered in English. We are still transitioning from a non-ELF course, and therefore you will find symbols and comments in Spanish throughout the provided VHDL source code. Do not hesitate to ask the faculty if you are a non-native Spanish speaker and run into trouble with this.
 - Report any typo or mistake you may find and ask about anything you need, but mind that we will drop questions which are already answered in this Guide.
- ☐ The checkboxes throughout this Guide constitute the checklist you must complete before submitting your project.
-

Contents

1 Summary	2
2 Baseline MIPS	2
2.1 IO Data Memory Subsystem	3
2.2 Support for Exceptions	3
2.3 Performance Counters	4
3 Your preliminary ToDo list	5
4 Tasks	5
4.1 Adding New Instructions	5
4.2 Managing Structural and Data Hazards	5
4.3 Managing Control Hazards	6
4.4 Managing Performance Counters	6
5 Results and Deliverables	6
5.1 Testing	6
5.2 Project Report	7
5.3 Estimated Project Timeline	7
6 Grading Policy	7
7 Final Remarks	8
7.1 Instruction Format	8
8 Looking for Help	8

1 Summary

The aim of this project is **to learn how to work with a hardware description language**, in a professional simulation environment, based on testbench and cronograms. The case study is a 32b MIPS processor, with a 5-stage pipeline, like the one introduced in classes and lab assignments. This processor is integrated in a System on Chip (SoC), with the features described in Sec. 2.

The first part of the project consists in modifying the provided baseline MIPS to support the two instructions added in Lab Assignment #3, plus a new additional instruction. This implies to modify both the control unit and the data path (Sec. 4.1). It requires **to manage a new structural hazard**, which arises because the data memory does not always perform the required access in one cycle. We will work it out by halting the processor until the hazard is resolved (Sec. 4.2).

In a second step, you have to apply data forwarding to eliminate data hazards if possible, otherwise stalling the processor in the IF and ID stages (Sec. 4.2). The destination of forwarded data will always be the Execution stage.

Third, you have to deal with the possible **control hazards** (Sec. 4.3), and then manage a **set of counters** you will use to monitor performance (Sec. 2.3).

Once all the former modifications work properly, you have to check out the **management of exceptions** (elements, operation, and management) as provided in the MIPS sources.

The project also **entails the design of testbenches and test programs** to check each step (unitary tests) and the overall design (integration test), see Sec. 5.1. Finally, you have to **draw up a report following the specifications** provided in Sec. 5.2.

The project has been conceived to be carried out in **pairs**. Greater groups are not permitted (Sec. 6).

2 Baseline MIPS

The provided baseline MIPS as the starting point of the project implements the MIPS processor explained in classes, more detailed than the one employed for the compulsory lab assignments. Its specific features are as follows:

- It implements a fully delayed MIPS ISA: the hardware does not manage any kind of hazards (i.e., the pipeline never stalls, and there are no resources to manage data forwarding). Any code run on this core as it comes out-of-the-box must avoid hazards leveraging static instruction scheduling, e.g., reordering instructions or inserting *nops*.
- Register *r0* works as any other register.¹
- Branches are resolved in the ID stage.
- Register writes are rising-edge triggered, but for those in the register file (RF), which is falling-edge triggered.
- Supports exceptions (Sec. 2.2).
- Provided as a core integrated in a *System-on-Chip* (SoC, check file `AOC2_SoC.vhd`), along with an IO data memory subsystem (Sec. 2.1).
- It features a new control signal (`f_inc`), set when the decoded instruction is a `lw_inc` (Sec. 4.1).

2.1 IO Data Memory Subsystem

Described in `IO_MD_subsystem.vhd`. It encompasses the MIPS data DRAM and a few memory-mapped IO registers (reachable using `lw/sw` instructions). Two of them are used to receive and send external data, and one for interrupt management. The DRAM is encapsulated into a component with support for the atomic instruction `lw_inc` (Sec. 4.1). A set of signals constitute the address bus, the data bus, and the control bus. The MIPS core and the IO data memory subsystem interact using three key signals:

- **IRQ:** See Sec. 2.2. It points out that there is an incoming interrupt request. The only interrupt source in this system is the signal (`Ext_IRQ`), which the `IO_MD_subsystem.vhd` forwards to the MIPS core using `MIPS_IRQ`. The core can acknowledge the interrupt writing 1 into the 1-bit internal register `ACK_REG`. This value is automatically reset to zero at the next cycle, to ease the process of acknowledgment. The content of `ACK_REG` is reachable through the output signal `INT_ACK`.
- **Data_Abort:** See Sec. 2.2. Indicates an illegal memory access (to an address which is either unaligned or out-of-range). To help with debugging, the signal will take the value `0xBAD0ADD0` when raised.
- **IO_Mem_ready** is set when the DRAM is ready to perform the required access in that cycle. This means that the core's control unit must stall the pipeline when a memory instruction (`lw`, `lw_inc`, `sw`) reaches the MEM stage and `IO_MEM_ready` = 0. To make debugging easier, reading the data bus while the DRAM is not ready will return the value `0x000BAD00`.

The IO data memory subsystem also includes two 32b registers (`IO_input`, `IO_output`) to receive or send values into/out of the system.

2.2 Support for Exceptions

Tab. 1 shows the four possible events causing an *exception* in the MIPS core of this project. The right column shows the interrupt vectors.²

The processor relies on the following resources to manage these exceptions:

- Status register (`status_reg`, 2 bits):
 - Setting (1) or clearing (0) the most significant bit (1), respectively, enable or disable exceptions.
 - The value of the least significant bit is 0 in normal mode, 1 when entering exception mode.
- Exception link register (`Exception_LR`): stores the instruction address to which the exception routine returns once it is over.

¹In the original MIPS ISA, a `lw` on *r0* always returns 0, and a `sw` on *r0* yields no effect.

²Data memory addresses which contain the address of the exception service routine to which the processor jumps upon each specific exception.

Event	Exception	Origin	Vector @
Reset	<i>reset</i>	External (testbench)	0x0
Interrupt request	<i>irq</i>	IO memory subsystem (signal <code>IRQ</code> = 1)	0x4
Reference to an invalid data memory address	<i>data abort</i>	Data_Memory_Subsystem (signal <code>Data_abort</code> = 1)	0x8
Unknown opcode	<i>undef</i>	UC	0xC

Table 1: Exceptions in the MIPS core of this project

- Mux to select the proper return address depending on the instruction we must resume to keep the program order: instruction in **EX**, **ID**, or **IF**. A specific signal propagated with the instruction down the pipeline indicates if an instruction is valid or invalid, and the register banks of stages **ID** and **EX** include a register to hold the address of the instruction in that stage.
- Additional entries in the **PC** mux to load the address of the proper exception service routine (stored in the corresponding exception vector) or the exception return address (**Exception_LR_output**).

The MIPS core takes the following actions when exceptions are enabled and one of the events in Tab. 1 occurs:

1. The instructions in **MEM** and **WB** proceed and terminate normally, whereas the instructions in **IF**, **ID**, and **EX** are cleared (study the code).
2. The return address of the oldest valid instruction in the pipeline is stored in **Exception_LR**.
3. The address of the exception service routine is copied from the proper offset in the exception vector into the **PC**. E.g. `PC <= x"00000004"` in the case of an **IRQ**.
4. The status register switches to exception mode, disabling (all types of) exceptions, **Status** <= "11".

The only exception which should actually allow the program to resume execution is *irq*, excuting the instruction **rte** (Sec. 4.1).³

2.3 Performance Counters

To analyze the performance of a program on a specific microprocessor we leverage tools known as *profilers* (e.g., *perf*). Nowadays, *profilers* rely on event counters integrated in the microprocessors. Our MIPS core must support the following event counters:

- Cycles since last reset.
- Dynamic instruction count: valid instructions completing in **WB**.
- Stall cycles due to data hazards.
- Stall cycles due to control hazards.
- Stall cycles due to memory references.
- Total number of accepted exceptions.

³The exception service routines for *data abort* and *undef* should halt the processor (which is what we do in our tests, for simplicity) or pass the control to the operating system. They can use **Exception_LR** to report about the error causing the exception.

3 Your preliminary ToDo list

- ☐ Download the companion materials from Moodle (source files, test, examples).
- ☐ Watch the video tutorials.
- ☐ Study the source code to understand the structure and function of the SoC and its components (MIPS and IO Memory subsystem). Identify the components introduced in the theory classes, problems, and Lab Assignment #3.
 - *Scratching your own schematics of the SoC and the MIPS will be definitely useful to you. Write down the names of the principal signals connecting the main components.*
- ☐ Create a project in the VHDL simulation environment you used in Lab Assignment #3. Compile and simulate the example test. Organize your own waveform of the MIPS and save it. Check out how the signals you see in your waveform evolve, according to instructions in the example test.
 - ☐ Track signal propagation through the register banks of consecutive stages.
 - ☐ Pay special attention to how exceptions work (control transfer, mode switching, `rte` instruction).
 - ☐ Check out the values written in the register file, data memory, and IO output.

4 Tasks

The baseline MIPS provided in the source code does not manage hazards and never stalls the pipeline (Sec. 2). Therefore, it cannot correctly execute programs compiled for non-delayed ISA.⁴ You have to make the due changes to support a non-delayed MIPS ISA, detecting structural, data, and control hazards, applying data forwarding when possible, and stalling the pipeline when required. It should correctly execute three new instructions.

4.1 Adding New Instructions

Design and apply the required modifications to support the following instructions:

- ☐ `jal` and `ret`, following the same semantics as in Lab Assignment #3.
- ☐ `lw.inc rt, imm(rs)` (*Atomic read-increment*), which performs the following actions:

```
rt <= Mem(rs + SignExt(imm))
Mem(rs + SignExt(imm)) <= Mem(rs + SignExt(imm)) + 1
PC <= PC + 4
```

- ☐ Read the component `RAM_128_32` of the data memory to understand the automaton (process) which control the in-memory increment.
- ☐ Leverage signal `f_inc` (mentioned in Sec. 2) to control the execution of `lw.inc`. This signal must be propagated down to the MEM stage using the expansion ports of the register files of stages EX and MEM.

4.2 Managing Structural and Data Hazards

You must complete the two following components:

- ☐ **Forwarding Unit** instantiated as `UA` (*Unidad de Anticipación*). The UA controls the two muxes you can already see at the ALU entry in the source code of the baseline MIPS. You must design the logic to check if the last value of the operands of a consumer instruction in stage ID must be taken from the RF or from the forwarding buses. Take special care with the operands of the new instructions. E.g., can we forward the value produced by `jal` in WB to a consumer at distance 1 leveraging our forwarding buses as specified? Do you think it is worth it to add specific resources in this processor to optimize this case?

⁴Therefore, producer and consumer instructions must be separated by independent instructions (`nop` if there are not any), and `beq`, `jal`, and `ret` do also require static scheduling, inserting `nop` if needed.

- **Hazard Detection Unit** instantiated as UD (*Unidad de Detención*). It looks for data hazards that cannot be overridden by data forwarding (e.g., load-use dependencies, and dependencies involving consumers which reads their operands in ID, such as `beq` or `ret`). Upon a data hazard, you must set the signal `stall_ID` to stall ID and IF until the hazard is over. Stalling ID implies making invalid the instruction to be sent to EX (i.e., unset `valid_I_IF`). UD is also in charge of memory stalls. If the memory is not ready (check `IO_MEM_ready`) you must set `stall_MIPS`. This will stall all the stages until the IO memory subsystem signals that it is ready. Mind that `stall_MIPS` disables exceptions too. However, `stall_ID` is irrelevant for exceptions, because when we jump to the exception service routine we will just kill the instruction in ID.

Caveats

- **Mind the validity bit** when enabling forwarding or stalling stages. It does not make any sense to forward data to an invalid instruction, much less stalling the pipeline.
- **Do only stall upon a hazard.** Consider which operands (`rs` y `rt`) uses each instruction and which instructions write the RF, including the new ones. Thus:
 - `nop` instructions do not generate data dependencies nor, consequently, data hazards. Stalling on a `nop` will be considered a minor mistake.
 - A `sw` instruction (which uses `rt` as its source) does not write the RF: it is not a producer on the RF, and cannot rise load-use hazards. Incorrectly stalling on a `sw` will be considered a minor mistake.
- Check and understand the uncompleted code we provide in the UD for asserting the usage of instructions' operands and validity. Make the necessary changes to include the cases arising from the new instructions.

4.3 Managing Control Hazards

All control transfers in the provided baseline MIPS are delayed, because the processor does not manage any hazards. Now, you must modify the UD to support the management of control hazards. There are four control transfer instructions (`beq`, `jal`, `ret` and `rte`). While these instructions are in the ID stage, the instruction in PC+4 is being fetched from memory in IF.

- `jal`, `ret`, and `rte` are unconditional jumps, and therefore the instruction in IF must always be invalidated (clearing `valid_I_EX_in`).
- In the `beq` instruction, the instruction in IF is only invalidated if the branch is taken (T.):⁵

4.4 Managing Performance Counters

Performance counters are disabled by default, except for the total cycle counter.

- Assign the proper value to their enabling signals so that they correctly record their corresponding events.
- Upon stalls, you must only enable one of the counters. For example, if there is a data hazard which implies a stall, and there is also a memory stall lasting four cycles, these four cycles add up to the memory stall counter, not to the data stalls counter. Once the memory becomes ready, we will increment the data stall counter if the data hazard persists.

5 Results and Deliverables

5.1 Testing

- Testing is what makes the difference between an Engineering project and a total blunder.
- There are two non-exclusive ways for testing this project: testbenches and test programs. We provide a base testbench to streamline your work. However, you are in charge of performing unitary tests (spotting specific functionalities and cases), and integration tests to check that all the features required in Sec. 4 work correctly.

⁵This is tantamount to say that we are predicting `beq` as not taken (NT).

– We evaluate the quality of your tests. Take them seriously.

- When you create new programs or modify an existing one you must codify the instructions in hexadecimal and write them into the instruction RAM component. Also, edit the data RAM if your modifications use new or modified variables or constants.

Testing ToDo List

- ☐ Place your test programs inside a single `.vhd` file for all the instruction RAMs, as we do in the instruction RAM file we provide (`RAM_I_test_exceptions.vhd`). Check that only one instruction RAM component remains un-commented (active) before compiling the project for simulating each test. Do not forget to make changes in the data RAM if required.
- ☐ Un-comment/comment the proper `stim_proc` as indicated in the source comments of the provided testbench, in order to check that your new instructions work correctly in the absence of interrupts. You can start leveraging the test program you used for Lab Assignment # 3, where your processor managed no hazards, with specific modifications as unitary tests.
- ☐ Go through all the test programs we include in (`RAM_I_test_exceptions.vhd`) until all your modifications pass all of them.
- ☐ Simulate the test program delivered with the sources, activating IRQs in the testbench (using the `stim_proc` which triggers irqs in the testbench).
- ☐ Simulate your test programs to fine-tune on every potential problem you might spot.

5.2 Project Report

- ☐ Check that you have fulfilled all the sections requested in the report template included in the companion materials. You can use the template in a straightforward way or adapt it to your preferences, but the original sections are mandatory.

5.3 Estimated Project Timeline

1. Preliminary ToDo list (Sec. 3): 2 h.
2. Adding the new instructions: 2 h.
3. Hazard management: 2 h.
4. Testing and debugging: 10 h.
5. Project Report: 3 h.

This estimate assumes you have been timely tracking the course (classes, problems, lab assignments), reasonably understanding the concepts, and have thoroughly read this Guide.

- ☐ Track the time you devote to the project. A simple spreadsheet can do the job.
- ☐ Compare the result with our estimate and write it down into the corresponding section of your report.

6 Grading Policy

We download, check, and simulate each submission using our own evaluation test, classifying submissions into three groups:

- S1: The submitted report is okay and the project passes our simulation test.
- S2: The project fails our simulation test but passes your tests, which are decently designed and explained, and the failures can be reasonably fixed.
- S3: None of the above.

Then, we open time slots for in-person interviews. We can ask you about any project detail (contents of your report, and what any signal or component or VHDL expression does or how it works). We can ask you to draw a chronogram on a specific instruction sequence, or ask questions about key concepts. We ask you individually despite you have worked in pairs. Both of you must prove a thorough understanding of all aspects of the project.

A good project report greatly helps during the interview.

Failure to demonstrate a thorough understanding of the project during the individual assessment may result in a failing project, even if classified as S1. Experience tells us that this is unlikely to happen: as a rule, you work very well and prove a good command of your project. Cases in S2 that pass the interview will be able to fix the issues and re-submit within 24 h.

7 Final Remarks

7.1 Instruction Format

You will fail the project if you use instruction formats different from the ones established in the baseline MIPS (see the slides for Unit 2) and this Guide.

Please find a spreadsheet in the companion materials which will help you codify the instructions.

- Operation codes (opcode field):
 - NOP: 000000; Arit: 000001; LW: 000010; SW: 000011; LW_inc: 010000
 - BEQ: 000100; JAL: 000101; RET: 000110; RTE: 001000
- Function codes for ALU operations (*funct* field and signal *ALU_ctrl1*):
 - +: 00000; -: 00001; AND:00010; OR: 00011

Data and instruction memories (*RAM_D_test.vhd*, *RAM_I_test.vhd*) are defined in VHDL signals of type *array* with 128 words of 32 bits.

Each word in the VHDL RAM is a 32-b word codified in hexa (8 nibbles, e.g. `X"88010000"`, `X"00000000"`...). Memory addresses are 32b wide, but we only consider 7 bits to address the RAM as an array of 128 words. Read thoroughly the VHDL code and find the relationship between memory address and word number in the array, so that you can easily convert back and forth effective address in a *lw/sw* instruction and the word number in the VHDL array (which is the one you will see at the simulator's waveform).

- We are referencing words, not bytes. Addressing an unaligned byte (i.e., a byte in a memory address which is not a multiple of 4) constitutes an invalid access and triggers a *data abort* exception (Sec. 2.2).

8 Looking for Help

- Each pair team must completely develop their own code. Taking code from others on a (near-) copy-paste basis is considered plagiarism. This does not exclude collaboration: you are encouraged to share ideas, discuss solutions together, and help your classmates.
- Do not hesitate to ask us for help, but the time we devote to these projects is not officially computed anywhere anymore. Therefore:
 - Be as specific as possible in your emails.
 - Check in Moodle the document *Project troubleshooting*. We update this document continuously for the whole of the course period.
 - We greatly appreciate requesting office hours (online or in-person). This allows us to better assess your project and provide constructive feedback that will ultimately benefit your grade.
 - Mind that our availability decreases non-linearly as the project's deadline approaches.